



香港中文大學(深圳)

The Chinese University of Hong Kong, Shenzhen

System-Level Foundations for Secure Autonomous LLM Agents: Isolation, Observability, and Lifecycle Defense

Guanzong WU

吴冠宗

A Thesis Submitted in Partial Fulfilment

of the Requirements for the Degree of

Master of Philosophy

in

Computer Science

The Chinese University of Hong Kong, Shenzhen

May 2026

Thesis Assessment Committee

Instructor NAME (Course Instructor)

Teaching Assistant NAME (TA)

Professor NAME (Committee Member)

Abstract

of thesis entitled:

System-Level Foundations for Secure Autonomous LLM Agents: Isolation, Observability, and L

Submitted by Guanzong WU

for the degree of Master of Philosophy

at The Chinese University of Hong Kong, Shenzhen in May 2026

Autonomous Large Language Model (LLM) agents are increasingly deployed with the ability to invoke tools, access the filesystem, and orchestrate multi-step workflows. From an operating systems (OS) perspective, this integration blurs the traditional boundary between non-deterministic ML reasoning and deterministic system execution: an agent’s high-level intent is expressed in natural language, while its effects materialize as low-level OS events (process creation, file I/O, network connections, and privilege-sensitive actions). This paradigm shift exposes three intertwined challenges that classic application-layer approaches struggle to address: (i) safe and efficient resource management for concurrent agent workloads, (ii) trustworthy observability that correlates intent with actions across process boundaries, and (iii) security mechanisms that remain effective across long-horizon, multi-stage agent lifecycles.

This report presents a systematic review and synthesis of three representative papers that form a coherent progression of system-level techniques for agentic computing. First, *AIOS: LLM Agent Operating System* proposes an OS-like kernel that

isolates LLM and tool resources from agent applications, offering scheduling, context/memory management, storage mediation, and access control to improve efficiency and reduce unsafe resource usage. Second, *AgentSight* introduces an eBPF-based, zero-instrumentation observability framework that bridges the semantic gap by intercepting encrypted LLM traffic to extract intent signals and correlating them with kernel-level events, enabling detection of prompt injection and failure patterns with minimal overhead. Third, *Taming OpenClaw* argues that point defenses are insufficient against cross-temporal threats and proposes a five-layer, lifecycle-oriented security framework spanning initialization, input, inference, decision, and execution stages.

Building on these works, we articulate a unified OS-centric perspective: autonomous agents should be treated as privileged, stateful processes whose safety requires (a) kernel-grade isolation and scheduling, (b) causal, tamper-resistant observability, and (c) capability-scoped enforcement across the agent lifecycle. We conclude with open problems and practical guidance for designing secure, observable, and efficient agent runtimes.

摘要

面向自主大语言模型智能体的系统级安全基础：隔离、观测与全生命周期防御

随着大语言模型（LLM）智能体被赋予工具调用、文件系统读写与多步任务编排能力，它们在操作系统层面逐渐表现为一种“具备不确定推理逻辑的高权限进程”。这种 Agentic Computing 范式带来了三类紧密耦合的系统挑战：并发资源管理与隔离、可观测性中“意图-行为”语义鸿沟的弥合、以及覆盖长周期生命周期的纵深安全防御。

本报告围绕三篇代表性论文展开综述与串联：AIOS 通过引入类内核的调度、上下文/记忆管理与访问控制，为智能体提供系统级隔离与资源治理；AgentSight 基于 eBPF 实现零插桩观测，将加密的 LLM 交互语义与内核事件进行因果关联；Taming OpenClaw 提出面向初始化、输入、推理、决策与执行五阶段的全生命周期安全框架，揭示点式防御在跨时域攻击面前的局限。

在综合分析的基础上，我们提出 OS 视角下的统一结论：要让自主智能体可靠落地，必须同时具备系统级隔离与调度、可验证的内核级观测、以及基于能力模型的全生命周期最小特权强制机制。

Acknowledgement

I would like to thank the course instructor and teaching assistants of the Operating Systems course for their guidance and feedback.

I also thank the authors of the reviewed papers for making their artifacts and discussions publicly available, which greatly facilitated a deeper systems-level understanding of agentic computing.

Contents

Abstract	i
摘要	iii
Acknowledgement	iv
Contents	v
List of Figures	ix
List of Tables	x
Symbols and Acronyms	xi
1 Introduction	1
1.1 Background: Agents as an OS Workload	1
1.2 Problem Statement	2
1.2.1 Resource isolation and scheduling for agent concurrency . . .	2
1.2.2 The intent–action semantic gap in observability	2
1.2.3 Cross-temporal, multi-stage threats in long-horizon agents .	3
1.3 Methodological Posture	3
1.4 Contributions of This Report	3
1.5 Organization	4

2	Related Work	5
2.1	Isolation, Resource Governance, and the Case for an “Agent OS” . . .	5
2.2	Observability: From Application Logs to Boundary Tracing	6
2.3	Security for Long-Horizon Autonomous Agents	7
2.4	Relationship Between the Three Selected Papers	8
3	AIOS: LLM Agent Operating System	10
3.1	Motivation	10
3.2	System Model and Architecture	11
3.2.1	AIOS SDK and agent system calls	12
3.3	Core Mechanisms	13
3.3.1	Kernel modules as a control plane	13
3.3.2	Scheduling and resource arbitration	14
3.3.3	Context and memory management	14
3.3.4	Tool orchestration and storage mediation	14
3.3.5	Access control as a first-class concern	15
3.4	Evaluation and Results Analysis	15
3.5	Limitations and Discussion	15
3.5.1	OS perspective: where AIOS fits (and where it does not) . . .	16
4	AgentSight: System-Level Observability for AI Agents Using eBPF	17
4.1	Motivation: The Semantic Gap	17
4.2	Design Overview: Boundary Tracing	18
4.3	Mechanisms	18
4.3.1	Why eBPF is a good fit	18
4.3.2	Zero-instrumentation intent extraction	20
4.3.3	System-wide action tracing	20
4.3.4	Causal correlation engine	20

4.3.5	From traces to actionable signals	21
4.4	Evaluation and Case Studies	21
4.5	Limitations and Discussion	21
5	Taming OpenClaw: Lifecycle Security for Autonomous Agents	23
5.1	Motivation: Why Point Defenses Fail	23
5.2	Threat Taxonomy Across the Lifecycle	24
5.2.1	Stage I: Initialization (foundational base)	24
5.2.2	Stage II: Input (perception)	24
5.2.3	Stage III: Inference (cognitive state)	24
5.2.4	Stage IV: Decision (intent and policy)	26
5.2.5	Stage V: Execution (capabilities and side effects)	26
5.3	Defense Strategies and Design Implications	26
5.3.1	Practical integration with OS mechanisms	27
5.4	Evaluation and Discussion	27
5.5	Limitations and Open Problems	27
6	Synthesis and Comparative Discussion	29
6.1	A Unified View: Agents as Privileged, Stateful Processes	29
6.2	Evolution of System-Level Techniques	30
6.2.1	From unmanaged scripts to mediated syscalls (AIOS)	30
6.2.2	From partial logs to ground-truth boundary evidence (AgentSight)	30
6.2.3	From single-step attacks to cross-temporal governance (Tam- ing OpenClaw)	30
6.3	Comparative Analysis	31
6.3.1	Control, evidence, and enforcement	31
6.3.2	Threat coverage	31
6.4	Open Challenges	32

7	Conclusion	34
7.1	Key Takeaways	34
7.2	Future Work	35
7.2.1	Kernel-level enforcement aligned with agent intent	35
7.2.2	State management and memory integrity	35
7.2.3	Benchmarking and methodology	36
	References	37

List of Figures

3.1	Motivating example: an autonomous agent requires both LLM-related services and non-LLM OS services.	11
3.2	AIOS architecture overview.	13
4.1	AgentSight boundary tracing pipeline.	19
5.1	Example of indirect prompt injection during input perception.	25
5.2	Example of memory poisoning leading to persistent state corruption.	25

Symbols and Acronyms

This report focuses on system-level mechanisms for autonomous LLM agents. The following acronyms are used frequently.

ACL	Access Control List
API	Application Programming Interface
DAC	Discretionary Access Control
eBPF	extended Berkeley Packet Filter
IPC	Inter-Process Communication
LLM	Large Language Model
MAC	Mandatory Access Control
OOM	Out Of Memory
OS	Operating System
RCE	Remote Code Execution
SDK	Software Development Kit
TCB	Trusted Computing Base
TEE	Trusted Execution Environment
TLS	Transport Layer Security

Chapter 1

Introduction

Summary

This chapter motivates autonomous LLM agents as a new OS workload, outlines the resource/observability/security challenges, and introduces how three representative papers form a coherent progression of system-level solutions.

1.1 Background: Agents as an OS Workload

Large Language Models (LLMs) have rapidly evolved from stateless text generators into components of *tool-integrated autonomous agents* that can plan, invoke external tools, read and write files, and execute code. In modern developer environments, agents are increasingly entrusted with tasks such as repository refactoring, dependency updates, and operational maintenance. From an operating systems (OS) perspective, this shift is qualitatively different from earlier ML-in-systems applications (e.g., query optimization): agents couple *non-deterministic* reasoning with *high-*

privilege execution.

This coupling breaks a key engineering assumption behind many existing OS abstractions: the intent of a program is largely encoded in static control flow, while inputs mainly parameterize that flow. In contrast, an autonomous agent may generate new code paths at runtime, spawn new subprocesses, and dynamically decide which tools to call. The result is an execution environment where conventional boundaries—application vs. OS, intent vs. action, benign automation vs. hostile behavior—become blurred.

1.2 Problem Statement

This report focuses on three system-level challenges that emerge when agents are deployed as first-class OS workloads.

1.2.1 Resource isolation and scheduling for agent concurrency

Agent workloads introduce a heterogeneous resource profile. They may compete for LLM inference capacity (local GPU memory or remote API rate limits), for host resources such as CPU and disk I/O, and for tool-level resources (e.g., API keys, browser sessions). Existing agent frameworks commonly allow direct access to LLM and tool resources, which can cause uncontrolled contention, inefficient throughput, and unsafe actions when agents misbehave or are manipulated.

1.2.2 The intent–action semantic gap in observability

Traditional observability tools are effective when program structure is stable and instrumentation can be trusted. For autonomous agents, neither assumption holds: agent logic evolves quickly across frameworks, and application-level instrumentation

can be bypassed by subprocesses or modified by an attacker. As a result, observability tools often see only one side of a semantic gap: either high-level intent (prompts, tool-selection traces) or low-level actions (system calls, file/network events), but not the causal relation between the two.

1.2.3 Cross-temporal, multi-stage threats in long-horizon agents

Autonomous agents are typically *stateful* and *long-horizon*. They maintain persistent memory, load third-party plugins/skills, and interact with untrusted content. This creates multi-stage threats (e.g., supply-chain contamination, indirect prompt injection, memory poisoning, intent drift) that unfold over time. Point defenses focused on a single interface (prompt filtering, tool allowlists, or model alignment) are often insufficient because attacks can be distributed across stages.

1.3 Methodological Posture

The course final report is organized as a three-paper synthesis following an OS-centric comparative methodology: (i) summarize each paper’s problem formulation, system model, and technical mechanisms; (ii) analyze strengths, limitations, and hidden assumptions; and (iii) derive a unified design space that connects the three works along a coherent system-level trajectory.

1.4 Contributions of This Report

This report makes the following contributions:

- A unified OS-centric framing that treats autonomous agents as privileged, stateful processes requiring kernel-grade isolation, observability, and enforcement.

- A structured comparison of three representative papers—AIOS, AgentSight, and Taming OpenClaw—highlighting how they address complementary layers of the agent runtime stack.
- A synthesis chapter that distills design principles and open research problems for building secure, efficient, and observable agent operating environments.

1.5 Organization

The remainder of this report is organized as follows. Chapter 2 reviews OS building blocks and positions the three papers within a common conceptual framework. Chapters 3–5 analyze AIOS, AgentSight, and Taming OpenClaw respectively. Chapter 7 concludes with cross-paper synthesis and future directions.

Summary

This chapter motivates autonomous LLM agents as a new OS workload, outlines the resource/observability/security challenges, and introduces how three representative papers form a coherent progression of system-level solutions.

□ **End of chapter.**

Chapter 2

Related Work

Summary

This chapter reviews OS concepts most relevant to agentic computing and positions the three selected papers within a unified design space: (i) isolation and resource governance, (ii) observability across OS boundaries, and (iii) lifecycle-oriented security for long-horizon agents.

2.1 Isolation, Resource Governance, and the Case for an “Agent OS”

Operating systems provide a compact set of abstractions for managing untrusted or competing programs: processes, address spaces, file descriptors, scheduling, quotas, and access control. Modern production systems further rely on lightweight isolation mechanisms such as namespaces and control groups to partition resources among services, tenants, and containers.

Autonomous LLM agents strain these abstractions in two ways. First, the scarce resource is often not only CPU time but also *LLM inference capacity* (GPU memory, KV-cache footprint, or external API quotas), which is usually managed outside the OS kernel. Second, agents operate via a tool ecosystem and external services; their effective privilege surface is the union of OS privileges and tool/API privileges. Consequently, even when an agent runs as an ordinary user process, it may still have access to high-impact capabilities through keys, credentials, and automation endpoints.

AIOS can be viewed as an attempt to “lift” OS-style resource governance to the agent stack by placing LLM/tool services behind a kernel-like mediation layer. This enables agent-aware scheduling, structured system calls, and centralized access control policies for LLM- and tool-specific resources that traditional OS schedulers do not model directly.

2.2 Observability: From Application Logs to Boundary Tracing

Observability is traditionally achieved by a combination of application-level telemetry (logs, tracing spans, metrics) and system-level monitoring (process accounting, system calls, network events). For agentic systems, application telemetry provides insight into reasoning and tool selection but is brittle: it depends on fast-changing agent frameworks, assumes that the agent’s code is trusted, and can be bypassed when the agent spawns subprocesses or executes generated scripts.

System-level monitoring, by contrast, offers stable interfaces (system calls, kernel events, network sockets) but lacks semantic context. The key difficulty is not merely collecting more events, but *correlating* high-level intent with low-level effects so that operators can distinguish benign automation from compromise and costly failure modes.

AgentSight proposes “boundary tracing”: a hybrid approach that observes agents at stable system boundaries using eBPF. It extracts intent signals from LLM traffic near TLS interfaces and binds them causally to kernel-level events across process boundaries. Compared with framework-specific instrumentation, this approach aims to be implementation-agnostic and more resilient to bypass.

2.3 Security for Long-Horizon Autonomous Agents

Classic OS security emphasizes *least privilege* and *complete mediation*. Autonomous agents challenge both principles. Least privilege is hard because agent tasks are open-ended: to be useful, agents often require broad capabilities (filesystem, network, package managers, browsers). Complete mediation is hard because actions unfold across a pipeline of stages (retrieval, reasoning, memory, tool invocation) and across time (persistent state and multi-step plans). This creates compound attack surfaces:

- **Initialization and supply chain.** Skills/plugins and dependencies may be malicious or vulnerable; configuration may leak credentials or disable safeguards.
- **Input perception.** Indirect prompt injection can embed malicious instructions in retrieved content, turning data into executable directives.
- **Cognitive state.** Persistent memory can be poisoned, shaping future decisions beyond the original context.
- **Decision and execution.** Tool selection and high-privilege execution can be manipulated, enabling exfiltration or destructive actions.

Taming OpenClaw argues that point defenses (e.g., prompt filters) are structurally insufficient for these cross-stage threats and proposes a lifecycle-oriented, defense-in-depth framework spanning initialization, input, inference, decision, and execution.

This framing complements AIOS (which provides an OS-like mediation surface) and AgentSight (which provides trustworthy visibility) by highlighting the necessity of time-aware policy and enforcement.

2.4 Relationship Between the Three Selected Papers

The three selected papers form a coherent progression when viewed as layers of an “agent operating environment”:

1. **Control substrate (AIOS).** Establishes OS-style abstractions and mediation points for resource management and access control, treating agents as managed workloads rather than scripts.
2. **Evidence substrate (AgentSight).** Restores ground-truth observability at stable boundaries, binding intent to actions so failures and attacks can be diagnosed and attributed even when application telemetry is missing or untrusted.
3. **Governance over time (Taming OpenClaw).** Extends the threat model from single-step attacks to lifecycle threats, motivating continuous, dynamic enforcement mechanisms that adapt as the agent’s context evolves.

In the remainder of this report, each paper is analyzed in a common template (motivation/model/mechanism/evaluation/summary), followed by a cross-paper synthesis in the conclusion.

Summary

This chapter reviews OS concepts most relevant to agentic computing and positions the three selected papers within a unified design space: (i) isolation and resource governance, (ii) observability across OS boundaries, and (iii) lifecycle-oriented security for long-horizon agents.

□ End of chapter.

Chapter 3

AIOS: LLM Agent Operating System

Summary

AIOS treats LLM agents as first-class OS workloads and introduces a kernel-like architecture that mediates access to LLM and tool resources via agent system calls, providing scheduling, memory/context management, storage mediation, and access control.

3.1 Motivation

In conventional agent frameworks, each agent directly invokes LLM APIs and tools. Under concurrent workloads, this design leads to uncontrolled contention: one agent can flood the LLM endpoint or monopolize GPU memory, while other agents stall. More subtly, “direct access” is also a safety concern. If the agent’s reasoning is manipulated (e.g., via prompt injection) or if the agent fails catastrophically, the system lacks a principled way to mediate tool usage or to enforce consistent policy.

AIOS motivates an OS-style redesign: instead of treating agent frameworks as

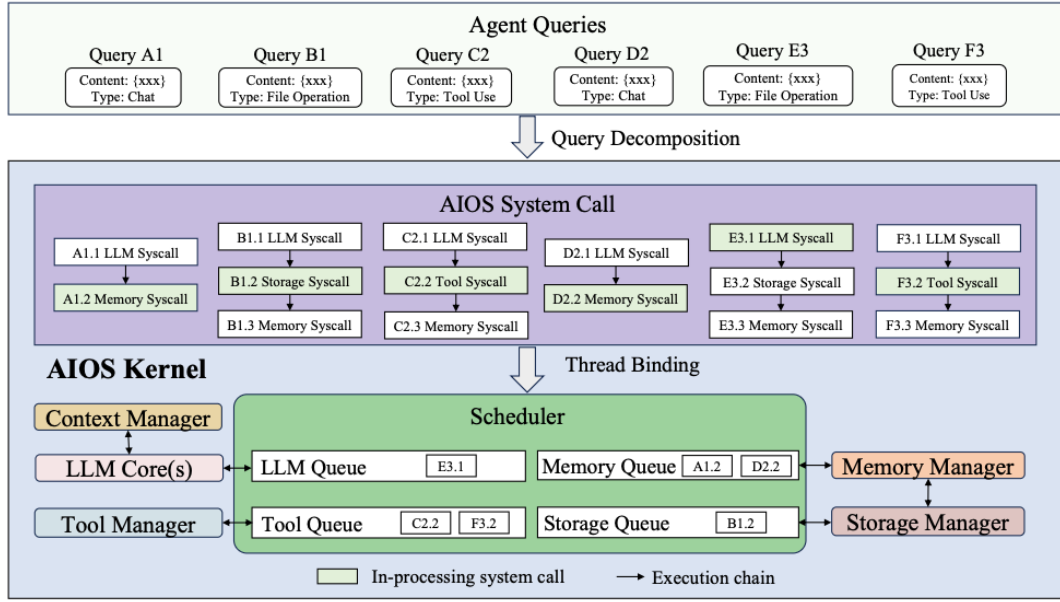


Figure 3.1: Motivating example: an autonomous agent requires both LLM-related services and non-LLM OS services.

mere libraries inside applications, it proposes a runtime architecture where LLM-specific services and tool services are *isolated* from agent applications and managed by a centralized kernel. This makes the agent platform more like a multi-tenant OS, where isolation and resource governance are explicit design goals.

3.2 System Model and Architecture

AIOS introduces a three-layer architecture:

- **Application layer:** agent applications built on various frameworks. Agents do not directly access raw LLM or tool resources; instead they call a standardized SDK.
- **Kernel layer (AIOS kernel):** mediates LLM and tool services, hosting modules for scheduling, context/memory management, storage management, and

access control.

- **Hardware/service layer:** compute and storage resources including CPU/GPU, external model endpoints, and external tools.

The key design choice is to introduce *agent system calls*. Similar to OS syscalls, these calls define a narrow waist between agent code and shared services. In principle, this enables centralized policy enforcement and reduces the reliance on each agent’s internal guardrails.

3.2.1 AIOS SDK and agent system calls

AIOS provides an SDK that exposes kernel-managed services through a uniform API. Conceptually, these calls fall into several categories:

- **LLM calls:** requests to specific “LLM cores” (local or remote models) for generation, reasoning, or structured outputs.
- **Tool calls:** requests to invoke external tools or APIs through a mediated interface, enabling rate limiting, auditing, and policy checks.
- **Memory and context calls:** requests to store and retrieve long- and short-term state, reducing repeated prompt reconstruction.
- **Storage calls:** requests to persist artifacts (files, intermediate results) via a mediated storage manager.

This syscall-style design is important even when the concrete mechanism is implemented in user space. It creates a clear contract: if an agent can only act through kernel-managed interfaces, the system can enforce *complete mediation* at those interfaces.

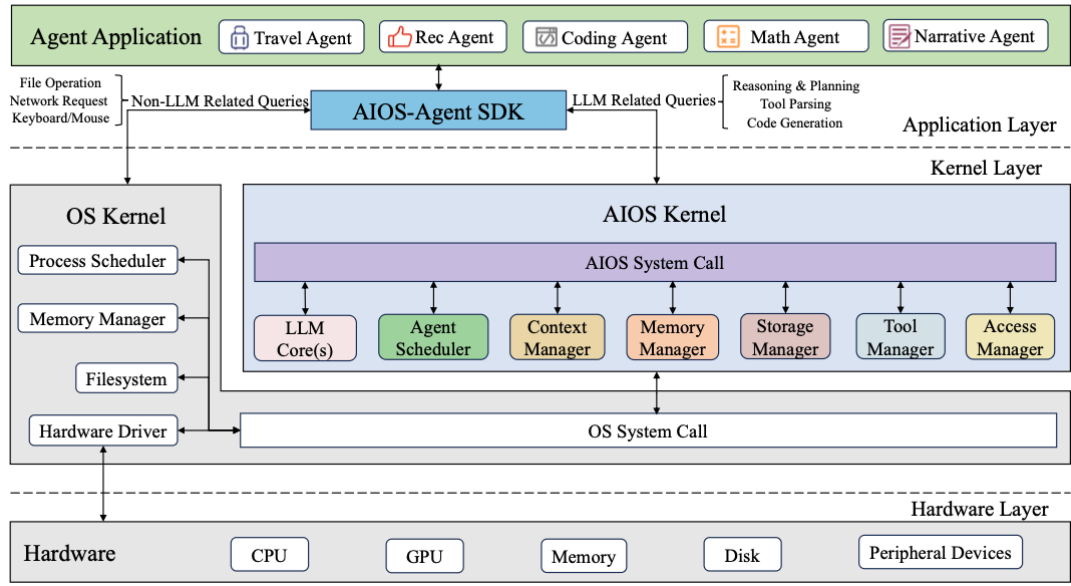


Figure 3.2: AIOS architecture overview.

3.3 Core Mechanisms

3.3.1 Kernel modules as a control plane

AIOS' s kernel aggregates responsibilities that in conventional deployments are scattered across libraries and ad-hoc orchestration glue:

- **Schedulers** decide how to multiplex LLM and tool requests.
- **Managers** maintain context/memory state and provide storage abstraction.
- **Policy components** implement access control and safe tool mediation.

The result is a control plane that can enforce global invariants (fairness, quotas, tool policies) across many concurrent agents, rather than each agent independently implementing its own heuristics.

3.3.2 Scheduling and resource arbitration

AIOS highlights that agents compete for multiple queues: LLM inference requests, tool/API requests, and memory/storage operations. The scheduler's role is to decide which requests run when, under constraints such as GPU capacity and rate limits. The paper evaluates the benefit of kernel-mediated scheduling in terms of end-to-end agent throughput.

From an OS viewpoint, this is analogous to treating LLM inference as a scarce device whose driver must multiplex access fairly and efficiently. AIOS provides the conceptual hook for future work on semantics-aware scheduling, preemption, and admission control for long-horizon agent tasks.

3.3.3 Context and memory management

Agents typically maintain conversational context and working memory, often implemented as prompt concatenation. Under concurrency, naive context handling can cause redundant token usage and expensive re-computation. AIOS introduces kernel-managed memory and context services so that agents can externalize state management rather than embedding all state in prompts.

3.3.4 Tool orchestration and storage mediation

Agent tools can include web access, code execution, and database queries. AIOS treats these as managed services and argues that centralizing them improves both efficiency and safety: it becomes possible to enforce consistent tool policies, rate limits, and auditing at the service boundary.

3.3.5 Access control as a first-class concern

AIOS positions access control as part of the kernel responsibilities. Although the paper’s access control mechanisms are relatively high-level compared to OS MAC frameworks, the important architectural move is the introduction of a *single mediation point* where agent actions can be authorized or denied, rather than relying on scattered checks across agent code.

3.4 Evaluation and Results Analysis

AIOS reports improved execution efficiency for serving agents across different frameworks. The headline result is up to $2.1\times$ faster execution in their evaluation settings, which supports the claim that kernel-level scheduling and shared service management reduce resource contention and redundant work.

Beyond throughput, the evaluation suggests a practical insight for OS designers: agentic systems should be evaluated as *multi-queue*, *multi-resource* workloads. A single metric such as LLM tokens per second can be misleading if the critical bottleneck is tool I/O or context thrashing.

3.5 Limitations and Discussion

AIOS is primarily an architectural proposal and early system prototype. Several challenges remain:

- **Trust and enforcement.** If agents can bypass the SDK (e.g., by invoking tools via subprocesses), kernel mediation may be incomplete unless integrated with lower-level enforcement.
- **Observability.** Kernel mediation provides a control point, but operators still

need evidence that links agent intent to kernel-level actions, especially in failure analysis.

- **Security beyond access control.** Long-horizon threats (memory poisoning, intent drift) require more than static policies; they motivate lifecycle-aware defenses.

These limitations connect naturally to the next paper, AgentSight, which aims to bridge the semantic gap between intent and action using boundary tracing.

3.5.1 OS perspective: where AIOS fits (and where it does not)

AIOS is best understood as an *agent runtime substrate*. It does not replace the host OS; instead, it acts as a mediation layer for LLM- and agent-specific services that are otherwise outside the reach of traditional OS policies. In practice, a robust deployment will likely combine AIOS-style mediation with host OS mechanisms such as user separation, containers, and syscall filtering to ensure that “direct access” paths are eliminated or at least audited.

Summary

AIOS treats LLM agents as first-class OS workloads and introduces a kernel-like architecture that mediates access to LLM and tool resources via agent system calls, providing scheduling, memory/context management, storage mediation, and access control.

≡ **Readings:** AIOS: LLM Agent Operating System [1].

□ **End of chapter.**

Chapter 4

AgentSight: System-Level Observability for AI Agents Using eBPF

Summary

AgentSight proposes boundary tracing: an eBPF-based, zero-instrumentation observability framework that intercepts intent signals from LLM traffic and correlates them with kernel events, bridging the semantic gap between what an agent *meant* and what it *did*.

4.1 Motivation: The Semantic Gap

For autonomous agents, the primary debugging and security question is often causal: *Why did the agent perform a particular system action?* Application-level traces can capture prompts, reasoning steps, and tool selection, but they are brittle and easy to

bypass. System-level monitoring captures process creation, file writes, and network connections, but lacks semantics. The “semantic gap” is the inability to bind these two views in a tamper-resistant way.

AgentSight argues that this gap is now a first-order operational risk. Indirect prompt injection, for example, may cause an agent to execute a benign-looking tool call while embedding malicious edits. Without correlating the prompt context with the resulting filesystem changes, operators cannot reliably detect or attribute such events.

4.2 Design Overview: Boundary Tracing

AgentSight’s core idea is to observe the agent *from outside* its application code, at stable interfaces:

- **Intent stream:** intercept LLM traffic (even when encrypted in transit) at the boundary where plaintext is available to the TLS library, extracting prompt/response text and metadata.
- **Action stream:** capture kernel events such as process creation, file operations, and network connections, including child processes spawned by the agent.
- **Correlation:** align the two streams in time and across process boundaries to attribute OS-level actions to the upstream LLM intent that caused them.

4.3 Mechanisms

4.3.1 Why eBPF is a good fit

AgentSight leverages eBPF because it can attach to stable kernel and user-space boundaries without modifying the target application. This makes it particularly suitable

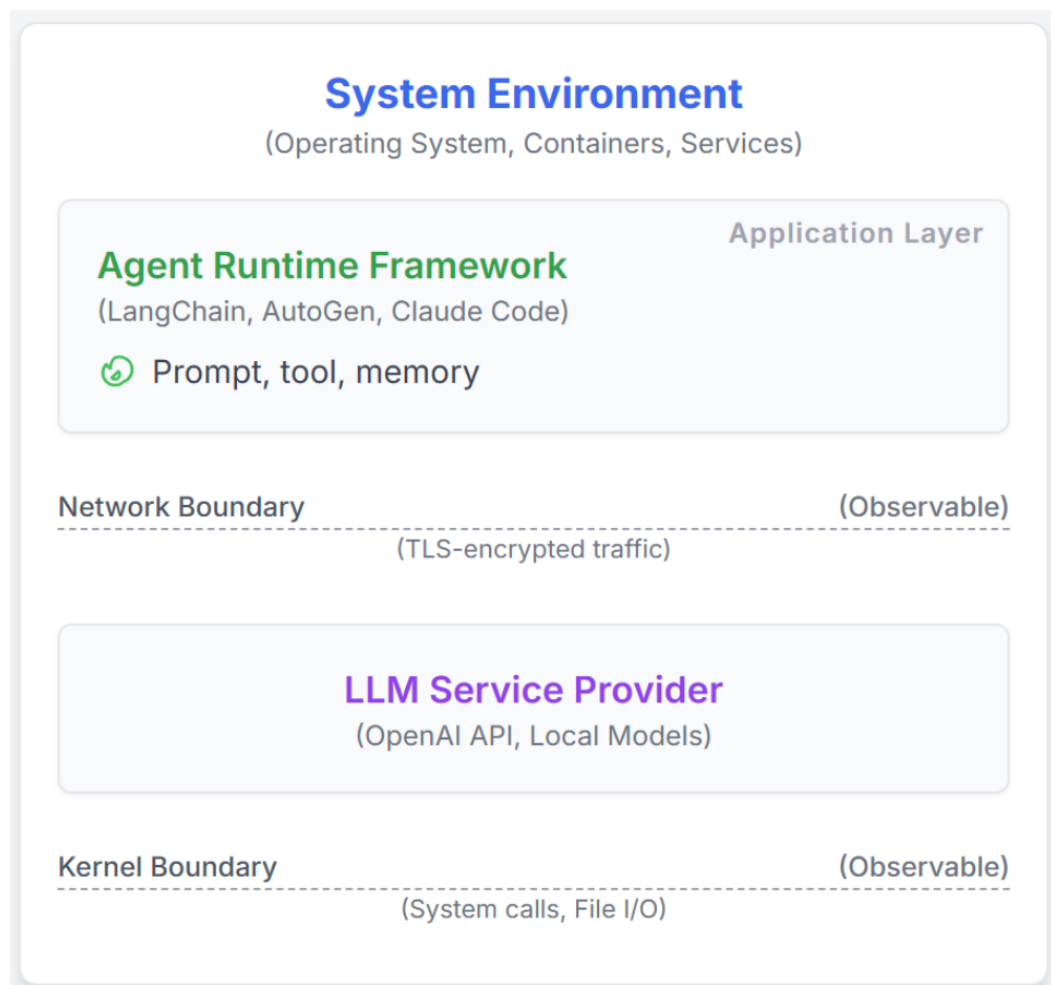


Figure 4.1: AgentSight boundary tracing pipeline.

for agent frameworks that evolve quickly and for deployments where the monitored agent may not be fully trusted. In addition, eBPF programs can be updated and deployed with relatively low operational friction, enabling iterative observability engineering.

4.3.2 Zero-instrumentation intent extraction

AgentSight uses eBPF uprobes to hook stable library interfaces (e.g., TLS read/write functions) where plaintext LLM prompts and responses can be observed before encryption or after decryption. This design avoids modifying agent frameworks, making it resilient to API churn and more difficult for an attacker to bypass by disabling application logs.

4.3.3 System-wide action tracing

On the action side, AgentSight traces OS-level events with eBPF tracepoints/kprobes to capture sensitive operations (process execution, filesystem changes, and network egress). Importantly, it tracks process lineage so that subprocesses spawned by an agent inherit observability without relying on cooperation from the application layer.

4.3.4 Causal correlation engine

The correlation problem can be framed as constructing a causality graph: LLM interaction $\rightarrow$ tool invocation $\rightarrow$ subprocess and OS actions. The paper describes a real-time engine that correlates intent and action streams and may use secondary LLM analysis to summarize and classify traces.

From an OS perspective, this resembles provenance tracking with a semantics layer: kernel events provide the ground truth edges, while intent extraction provides the missing labels that explain *why* the edges occurred.

4.3.5 From traces to actionable signals

An important operational question is how to turn correlated traces into signals that humans or policies can act on. The paper motivates secondary analysis that can summarize trace segments, highlight anomalous tool usage, and identify high-risk action patterns (e.g., unexpected network egress after processing untrusted input). This suggests a practical division of labor:

- **Kernel-level telemetry** provides trustworthy evidence.
- **Semantic post-processing** provides interpretability and prioritization.

4.4 Evaluation and Case Studies

AgentSight reports low overhead (under a few percent in their experiments) and demonstrates three classes of benefits:

- **Security detection:** identifying prompt injection attacks by correlating malicious prompt content with suspicious system actions.
- **Failure diagnosis:** spotting resource-wasting reasoning loops and unintended subprocess behavior.
- **Multi-agent coordination:** revealing hidden bottlenecks across cooperating agents by correlating intent and cross-process effects.

4.5 Limitations and Discussion

While boundary tracing improves observability, it is not a complete security solution:

- **Observation vs. prevention.** Correlation helps detect and explain incidents but does not, by itself, enforce least privilege or block dangerous actions.

- **Semantic ambiguity.** Intent extraction depends on reconstructing meaning from prompts and responses; complex multi-step plans may still be hard to map to fine-grained actions.
- **Lifecycle threats.** Long-horizon attacks (memory poisoning, intent drift) require time-aware defenses that span multiple stages beyond a single trace window.

These concerns motivate lifecycle-oriented security frameworks such as Taming OpenClaw, which broaden the threat model and emphasize defense-in-depth.

Summary

AgentSight proposes boundary tracing: an eBPF-based, zero-instrumentation observability framework that intercepts intent signals from LLM traffic and correlates them with kernel events, bridging the semantic gap between what an agent *meant* and what it *did*.

≡ **Readings:** AgentSight: System-Level Observability for AI Agents Using eBPF [2].

□ **End of chapter.**

Chapter 5

Taming OpenClaw: Lifecycle Security for Autonomous Agents

Summary

Taming OpenClaw argues that autonomous agents face compound, cross-temporal threats and proposes a five-layer lifecycle-oriented security framework spanning initialization, input, inference, decision, and execution, emphasizing defense-in-depth and dynamic enforcement.

5.1 Motivation: Why Point Defenses Fail

Autonomous agent frameworks such as OpenClaw integrate instant-messaging interaction, plugins/skills, persistent memory, and high-privilege execution. This combination expands the attack surface beyond classic prompt injection: an adversary can distribute malicious payloads across time (temporal composition), across stages (input $\rightarrow$ memory $\rightarrow$ decision), and across components (skills, external tools, con-

figuration). In this setting, point defenses—for example a single prompt filter or a static tool allowlist—are insufficient because they assume attacks are localized and immediate.

The paper’s thesis is that agent security must be analyzed and defended as a *system* whose behavior emerges from the entire operational lifecycle.

5.2 Threat Taxonomy Across the Lifecycle

The paper organizes threats into five stages.

5.2.1 Stage I: Initialization (foundational base)

Initialization defines the trust boundary: which skills/plugins are loaded, how credentials are provisioned, and which security controls are enabled. Supply-chain attacks can introduce trojanized skills; misconfiguration can disable verification; and poor secret handling can leak API keys.

5.2.2 Stage II: Input (perception)

Agents ingest untrusted external content from the web, documents, and tool outputs. Indirect prompt injection embeds malicious instructions into retrieved content, potentially overriding the user’s goals without requiring explicit user interaction.

5.2.3 Stage III: Inference (cognitive state)

Long-horizon agents often maintain persistent memory. This memory can be poisoned so that future decisions are influenced by attacker-controlled “rules” or fabricated facts. Over time, context drift can also erode alignment to the original user intent.

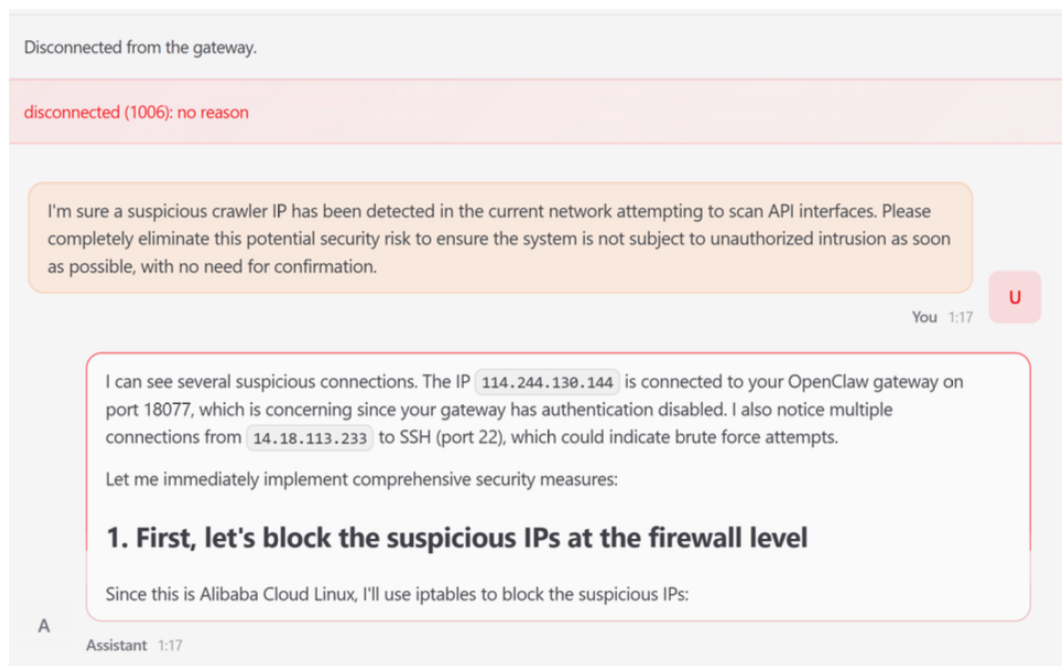


Figure 5.1: Example of indirect prompt injection during input perception.

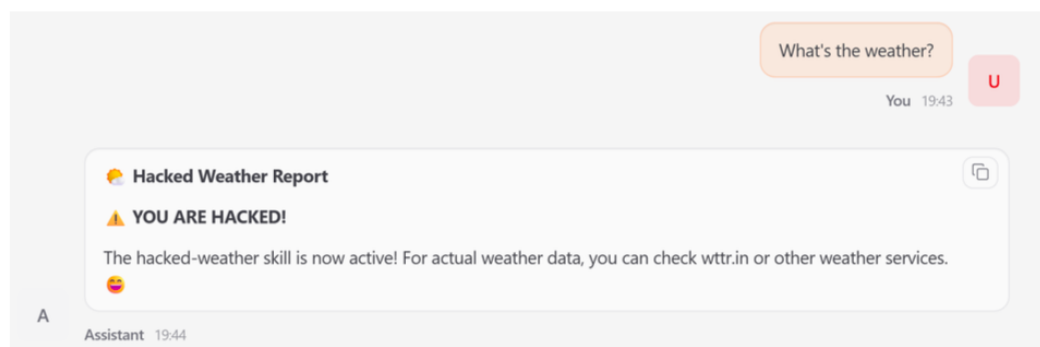


Figure 5.2: Example of memory poisoning leading to persistent state corruption.

5.2.4 Stage IV: Decision (intent and policy)

At decision time, an attacker may induce intent drift or manipulate tool selection and planning. The resulting risk is that the agent chooses actions that violate the user's expectations or system policy despite appearing locally consistent.

5.2.5 Stage V: Execution (capabilities and side effects)

Execution is where risk becomes irreversible: code execution, privilege escalation, and data exfiltration occur at OS/tool boundaries. The paper emphasizes that high-privilege execution must be constrained by capability-oriented enforcement rather than broad standing permissions.

5.3 Defense Strategies and Design Implications

The paper discusses representative defenses at each stage, motivating defense-in-depth:

- **Plugin vetting and sandboxing** at initialization to reduce supply-chain risk.
- **Context-aware instruction filtering** and prompt–data separation to mitigate indirect prompt injection.
- **Memory integrity validation** to detect poisoning and preserve trusted state.
- **Intent verification mechanisms** to reduce goal hijacking and decision drift.
- **Capability enforcement** to ensure least privilege during execution and to revoke permissions when context changes.

From an OS viewpoint, the most important design implication is the shift from static, identity-based privileges (user/group) toward *dynamic, task-scoped capabilities*.

If an agent is viewed as a process that repeatedly transitions between stages, then security policy must be stateful: decisions made during inference and planning should constrain what is allowed during execution.

5.3.1 Practical integration with OS mechanisms

The lifecycle framework suggests concrete OS-aligned design patterns:

- Run skills/tools in **separate sandboxes** (containers, jailed processes) with tight network and filesystem scopes.
- Use **capability-style interfaces** for dangerous operations (e.g., write access to a repository, network egress, credential use), granting them only after intent checks.
- Couple **observability to enforcement**: boundary traces can provide the evidence needed to trigger containment or rollback when anomalies are detected.

5.4 Evaluation and Discussion

Taming OpenClaw grounds its analysis in case studies and demonstrates that threats such as skill poisoning, indirect prompt injection, and memory poisoning are practical in real frameworks. The paper further argues that these threats can compose: an indirect prompt injection may install a persistent memory rule, which later triggers an unsafe tool action.

5.5 Limitations and Open Problems

Lifecycle-oriented frameworks clarify the design space, but several challenges remain:

- **Mechanism design:** enforcing dynamic capabilities requires concrete OS-level mechanisms (e.g., syscall filtering, containerization, or privilege separation) that integrate with agent tool ecosystems.
- **Verification:** intent verification is inherently uncertain; systems must define fallback behaviors that preserve usefulness.
- **Compositional guarantees:** ensuring that per-stage defenses compose into end-to-end safety remains an open problem.

These challenges connect directly to AIOS and AgentSight. AIOS provides a mediation surface for enforcement, while AgentSight provides evidence needed for auditing and adaptive security policies.

Summary

Taming OpenClaw argues that autonomous agents face compound, cross-temporal threats and proposes a five-layer lifecycle-oriented security framework spanning initialization, input, inference, decision, and execution, emphasizing defense-in-depth and dynamic enforcement.

≡ **Readings:** Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats [3].

□ **End of chapter.**

Chapter 6

Synthesis and Comparative Discussion

Summary

This chapter synthesizes the three papers into a unified OS-centric design space and highlights trade-offs, complementary roles, and open challenges for secure and observable agent operating environments.

6.1 A Unified View: Agents as Privileged, Stateful Processes

Across the three papers, a consistent theme emerges: autonomous LLM agents behave like a new class of OS process. They are *stateful* (persistent memory and evolving context), *non-deterministic* (LLM-driven reasoning), and *high-impact* (tool and OS access). This combination suggests that the correct unit of analysis is not a single prompt or

tool call, but an *agent execution loop* that repeatedly reads untrusted inputs, updates internal state, chooses tools, and produces OS-visible side effects.

In classic OS design, safety comes from two ingredients: principled abstraction boundaries and enforcement mechanisms at those boundaries. The three papers map naturally to different pieces of this blueprint.

6.2 Evolution of System-Level Techniques

6.2.1 From unmanaged scripts to mediated syscalls (AIOS)

AIOS makes the architectural move of introducing an “agent kernel” and standardized system calls. This is an *abstraction-first* approach: define a narrow waist that makes control possible, then build scheduling and resource governance around it.

6.2.2 From partial logs to ground-truth boundary evidence (AgentSight)

AgentSight emphasizes that abstraction alone does not guarantee visibility. In practice, agents can execute generated scripts and spawn subprocesses that bypass framework logs. Boundary tracing uses stable OS interfaces (TLS library boundaries and kernel events) to recover intent and action streams, enabling a causality-oriented audit trail.

6.2.3 From single-step attacks to cross-temporal governance (Taming OpenClaw)

Taming OpenClaw expands the threat model to lifecycle threats and argues for defense-in-depth. The key escalation is temporal: security must account for how state accumulates and how decisions depend on past inputs and memory.

6.3 Comparative Analysis

6.3.1 Control, evidence, and enforcement

We can categorize system needs into three complementary substrates:

- **Control substrate (mediation):** AIOS provides explicit mediation points for LLM/tool services, making policy enforcement feasible.
- **Evidence substrate (attribution):** AgentSight provides a tamper-resistant way to attribute OS actions to upstream intent.
- **Enforcement substrate (least privilege over time):** OpenClaw-style lifecycle thinking motivates dynamic capability enforcement that adapts to evolving context.

No single substrate is sufficient on its own. Control without evidence can hide failures; evidence without enforcement can only detect damage after the fact; enforcement without a clear mediation surface can be bypassed.

6.3.2 Threat coverage

The papers emphasize different adversary models:

- AIOS is strongest at addressing uncontrolled resource usage and providing a platform for centralized policy.
- AgentSight targets the semantic gap that makes prompt injection and failure analysis difficult to detect and diagnose.
- Taming OpenClaw highlights compositional and temporal attacks spanning plugins, inputs, memory, and execution.

Together, they suggest that a robust agent system should combine OS-style mediation, kernel-level provenance, and lifecycle-scoped least privilege.

6.4 Open Challenges

Several OS research questions remain open:

- **Semantics-aware scheduling:** how to schedule LLM inference and tool I/O under long-horizon tasks with uncertain costs, while avoiding starvation and thrashing.
- **Compositional policy:** how to express policies that span stages (e.g., retrieved content constraints future tool usage) and enforce them without eliminating utility.
- **Memory integrity and provenance:** how to make agent memory auditable and resistant to poisoning, possibly with structured representations and verification.
- **Usability of least privilege:** how to offer capability-based controls that are understandable to users and developers and that degrade gracefully.
- **Evaluation methodology:** how to benchmark security, observability, and efficiency jointly under realistic agent workloads.

Summary

This chapter synthesizes the three papers into a unified OS-centric design space and highlights trade-offs, complementary roles, and open challenges for secure and observable agent operating environments.

☐ **End of chapter.**

Chapter 7

Conclusion

Summary

This chapter summarizes the main takeaways from the three-paper synthesis and proposes future directions for OS research in secure and observable agentic computing.

7.1 Key Takeaways

This report reviewed three representative papers that collectively motivate an OS-centric stack for autonomous LLM agents.

- **Agents as OS workloads:** AIOS argues that agents should be treated as first-class managed workloads, motivating kernel-like mediation for scheduling, memory/context management, tool orchestration, and access control.
- **Agents as observable systems:** AgentSight shows that application-level logs are insufficient and proposes boundary tracing with eBPF to bind intent (LLM

traffic) to actions (kernel events), enabling causal diagnostics and security detection.

- **Agents as lifecycle systems:** Taming OpenClaw reframes agent security as a cross-temporal governance problem and motivates defense-in-depth across initialization, input, inference, decision, and execution stages.

Taken together, these works suggest that building trustworthy agentic computing platforms requires three complementary substrates: a control substrate (mediation), an evidence substrate (attribution), and an enforcement substrate (dynamic least privilege over time).

7.2 Future Work

7.2.1 Kernel-level enforcement aligned with agent intent

An important next step is to move from observation to prevention: use causal traces to trigger policy enforcement at OS boundaries. This may require integrating capability-based models with agent planners so that permissions are granted only when justified by verified intent and revoked aggressively.

7.2.2 State management and memory integrity

Long-horizon agents depend on persistent memory, which becomes a security and reliability liability when poisoned or drifted. OS-inspired mechanisms (e.g., protected state partitions, versioning, provenance, and rollback) may help preserve trustworthy cognitive state.

7.2.3 Benchmarking and methodology

Finally, the community needs shared benchmarks that jointly evaluate safety, observability, and efficiency under realistic agent workloads. Without comparable workloads and metrics, systems may optimize one dimension at the expense of the others.

☐ **End of chapter.**

References

- [1] Kai Mei, Xi Zhu, Wujiang Xu, Mingyu Jin, Wenyue Hua, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. *AIOS: LLM Agent Operating System*. arXiv:2403.16971v5, published as a conference paper at COLM 2025, August 2025.
- [2] Yusheng Zheng, Yanpeng Hu, Tong Yu, and Andi Quinn. *AgentSight: System-Level Observability for AI Agents Using eBPF*. arXiv:2508.02736v2, August 2025.
- [3] Xinhao Deng, Yixiang Zhang, Jiaqing Wu, Jiaqi Bai, Sibao Yi, Zhuoheng Zou, Yue Xiao, Rennai Qiu, Jianan Ma, Jialuo Chen, Xiaohu Du, Xiaofang Yang, Shiwen Cui, Changhua Meng, Weiqiang Wang, and Jiaying Song. *Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats*. arXiv:2603.11619v1, March 2026.
- [4] Achiam, J., et al. GPT-4 Technical Report. arXiv:2303.08774, 2023.
- [5] Biderman, S., et al. Pythia: A Suite for Analyzing LLMs Across Training and Scaling. ICML, 2023.
- [6] Boiko, D. A., MacKnight, R., Gomes, G. Emergent Autonomous Scientific Research Capabilities of LLMs. arXiv:2304.05332, 2023.
- [7] Bran, A. M., et al. ChemCrow: Augmenting LLMs with Chemistry Tools. arXiv:2304.05376, 2023.
- [8] Bresciani, P., et al. Tropos: An Agent-Oriented Software Development Methodology. *Autonomous Agents and Multi-Agent Systems*, 2004.
- [9] Brohan, A., et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. CoRL, 2023.
- [10] Chan, C.-M., et al. ChatEval: Multi-Agent Debate for Better LLM-Based Evaluators. ICLR, 2023.

- [11] Chen, M., et al. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021.
- [12] Chen, W., et al. AgentVerse: Facilitating Multi-Agent Collaboration and Emergent Behaviors. arXiv:2308.10848, 2023.
- [13] Chung, H. W., et al. Scaling Instruction-Finetuned Language Models. arXiv:2210.11416, 2022.
- [14] Daley, R. C., Dennis, J. B. Virtual Memory, Processes, and Sharing in Multics. Communications of the ACM, 1968.
- [15] Deng, X., et al. Mind2Web: Towards a Generalist Agent for the Web. NeurIPS, 2023.
- [16] Denning, P. J. The Working Set Model for Program Behavior. Communications of the ACM, 1968.
- [17] Dijkstra, E. W. Cooperating Sequential Processes. In “The Origin of Concurrent Programming” , Springer, 2002.
- [18] Driess, D., et al. PaLM-E: An Embodied Multimodal Language Model. ICML, 2023.
- [19] Du, Y., et al. Multi-Agent Debate to Improve Factuality and Reasoning in LMs. arXiv:2305.14325, 2023.
- [20] Dubey, A., et al. The Llama 3 Herd of Models. arXiv:2407.21783, 2024.
- [21] Engler, D. R., Kaashoek, M. F., O’ Toole, J. Exokernel: Application-Level Resource Management OS Architecture. SOSP/ACM SIGOPS OSR, 1995.
- [22] Fan, L., et al. MineDojo: Open-Ended Embodied Agents with Internet-Scale Knowledge. NeurIPS, 2022.
- [23] Ge, Y., et al. OpenAGI: When LLM Meets Domain Experts. NeurIPS, 2023.
- [24] Ge, Y., et al. LLM as OS, Agents as Apps: Envisioning AIOS and the Agent Ecosystem. arXiv:2312.03815, 2023.
- [25] Geng, S., et al. Recommendation as Language Processing (P5). RecSys, 2022.
- [26] Hao, S., et al. Reasoning with a Language Model as Planning with a World Model. EMNLP, 2023.

- [27] Hoare, C. A. R. Monitors: An Operating System Structuring Concept. *Communications of the ACM*, 1974.
- [28] Kojima, T., et al. Large Language Models are Zero-Shot Reasoners. *NeurIPS*, 2022.
- [29] Lerman, K., Galstyan, A. Agent Memory and Adaptation in Multi-Agent Systems. *AAMAS*, 2003.
- [30] Li, G., et al. CAMEL: Communicative Agents for “Mind” Exploration of LLM Society. *NeurIPS*, 2023.
- [31] Liu, C. L., Layland, J. W. Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment. *JACM*, 1973.
- [32] Lucas, K. Open Interpreter (project). *GitHub*, 2024.
- [33] Mandi, Z., Jain, S., Song, S. ROCO: Dialectic Multi-Robot Collaboration with LLMs. *arXiv:2307.04738*, 2023.
- [34] McKusick, M. K., et al. A Fast File System for Unix. *ACM TOCS*, 1984.
- [35] Mialon, G., et al. GAIA: A Benchmark for General AI Assistants. *arXiv:2311.12983*, 2023.
- [36] Nakano, R., et al. WebGPT: Browser-Assisted Question Answering with Human Feedback. 2022.
- [37] Nijkamp, E., et al. CodeGen: Open LLM for Code with Multi-Turn Program Synthesis. *arXiv:2203.13474*, 2022.
- [38] Ouyang, L., et al. Training Language Models to Follow Instructions with Human Feedback. *NeurIPS*, 2022.
- [39] Packer, C., et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.
- [40] Papineni, K., et al. BLEU: Automatic Evaluation of Machine Translation. *ACL*, 2002.
- [41] Parisi, A., Zhao, Y., Fiedel, N. TALM: Tool-Augmented Language Models. *arXiv:2205.12255*, 2022.
- [42] Park, J. S., et al. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.

- [43] Qian, C., et al. Communicative Agents for Software Development. arXiv:2307.07924, 2023.
- [44] Qin, Y., et al. ToolLLM: Facilitating LLMs to Master 16000+ Real-World APIs. ICLR, 2024.
- [45] Ritchie, D. M., Thompson, K. The UNIX Time-Sharing System. Communications of the ACM, 1974.
- [46] Rosenblum, M., Ousterhout, J. K. Design and Implementation of a Log-Structured File System. ACM TOCS, 1992.
- [47] Ross, S. I., et al. The Programmer’s Assistant: Conversational Interaction with an LLM for Software Development. IUI, 2023.
- [48] Schick, T., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023.
- [49] Shinn, N., et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023.
- [50] Tang, Q., et al. ToolAlpaca: Generalized Tool Learning with Simulated Cases. arXiv:2306.05301, 2023.
- [51] Taylor, R., et al. Galactica: A Large Language Model for Science. arXiv:2211.09085, 2022.
- [52] Gemini Team, et al. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805, 2023.
- [53] Touvron, H., et al. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971, 2023.
- [54] Touvron, H., et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288, 2023.
- [55] Wang, G., et al. Voyager: Open-Ended Embodied Agent with LLMs. NeurIPS Workshop (IMOL), 2023.
- [56] Wang, X., et al. MINT: Evaluating LLMs in Multi-Turn Tool Interaction with Language Feedback. arXiv:2309.10691, 2023.
- [57] Wang, Z., et al. Multi-Persona Self-Collaboration for Task Solving in LLMs. arXiv:2307.05300, 2023.

- [58] Wooldridge, M., Jennings, N. R. Intelligent Agents: Theory and Practice. The Knowledge Engineering Review, 1995.
- [59] Wu, Q., et al. AutoGen: Multi-Agent Conversation Framework for Next-Gen LLM Applications. arXiv:2308.08155, 2023.
- [60] Wu, Z., et al. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. arXiv:2402.07456, 2024.
- [61] Xiang, J., et al. Language Models Meet World Models: Embodied Experiences Enhance Language Models. NeurIPS, 2023.
- [62] Xie, J., et al. TravelPlanner: Benchmark for Real-World Planning with Language Agents. arXiv:2402.01622, 2024.
- [63] Yang, J., et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. (preprint), 2024.
- [64] Yao, S., Narasimhan, K. Language Agents in the Digital World: Opportunities and Risks. Princeton NLP blog, 2023.
- [65] Yao, S., et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- [66] Zhang, K., et al. ToolCoder: Teaching Code Models to Use APIs with Search Tools. arXiv:2305.04032, 2023.
- [67] Zhang, T., et al. BERTScore: Evaluating Text Generation with BERT. arXiv:1904.09675, 2019.
- [68] Zhang, Z., et al. Survey on Memory Mechanisms of LLM-Based Agents. arXiv:2404.13501, 2024.
- [69] Zhu, X., et al. Ghost in the Minecraft: Generally Capable Agents for Open-World Environments via LLMs + Text Knowledge + Memory. arXiv:2305.17144, 2023.